



Anna-Lena Popkes

April 13, 2021

Support vector machines

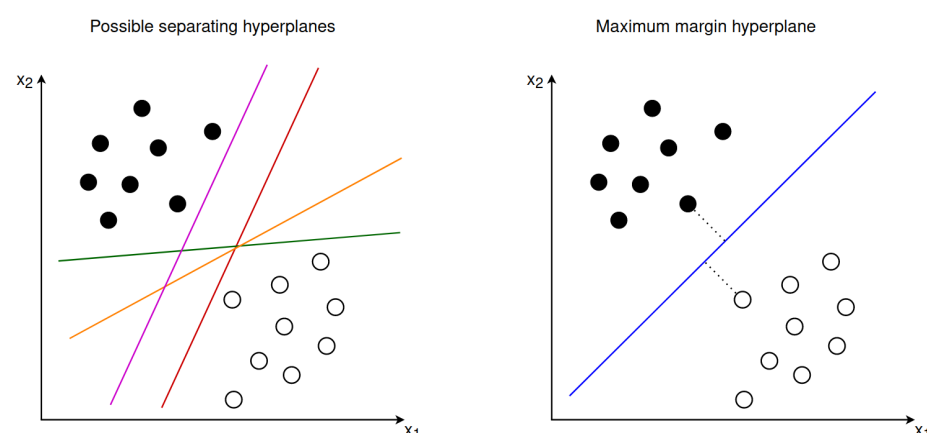
I posted another notebook in my [machine learning basics repository](#). This time, I took a detailed look at support vector machines. The blog post below contains the same content as the [original notebook](#). You can [run the notebook directly in your Browser using Binder](#).

1. What are support vector machines?

Support vector machines (short: SVMs) are supervised machine learning models. They are the most prominent member of the class of [kernel methods](#). SVMs can be used both for classification and regression. The original SVM proposed in 1963 is a simple binary linear classifier. What does this mean?

Assume we are given a dataset $D = \{\mathbf{x}_n, y_n\}_{n=1}^N$, where $\mathbf{x} \in \mathbb{R}^D$ and labels $y_n \in -1, +1$. A linear (hard-margin) SVM separates the two classes using a $(D - 1)$ dimensional) hyperplane.

Special to SVMs is that they use not any hyperplane but the one that maximizes the distance between itself and the two sets of datapoints. Such a hyperplane is called *maximum-margin* hyperplane:



In case you have never heard the term margin: the margin describes the distance between the hyperplane and the closest examples in the dataset.

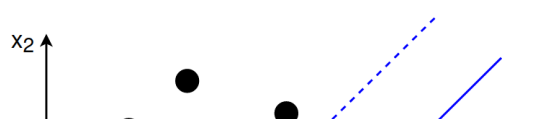
Two types of SVMs exist: primal SVMs and dual SVMs. Although most research in the past looked into dual SVMs both can be used to perform non-linear classification. Therefore, we will look at both approaches and compare them in the end.

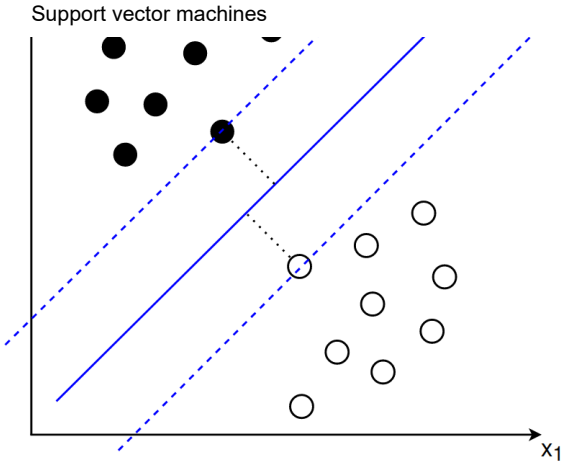
2. Primal approach - Hard-margin SVM

When training an SVM our goal is to find the hyperplane that maximizes the margin between the two sets of points. This hyperplane is fully defined by the points closest to the margin, which are also called *support vectors*.

The equation of a hyperplane is given by $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$. If an example $\mathbf{x}_i$ lies on the right side of the hyperplane (that is, it has a positive label) we have $\langle \mathbf{w}, \mathbf{x} \rangle + b > 0$. If instead $\mathbf{x}_i$ lies on the left side (= negative label) we have $\langle \mathbf{w}, \mathbf{x} \rangle + b < 0$.

The support vectors lie exactly on the margin and the optimal separating hyperplane should have the same distance from all support vectors. In this sense the maximum margin hyperplane lies between two separating hyperplanes that are determined by the support vectors:





2.1 Goal 1

When deriving a formal equation for the maximum margin hyperplane we assume that the two delimiting hyperplanes are given by:

$$\langle \mathbf{w}, \mathbf{x}_+ \rangle + b = +1$$

$$\langle \mathbf{w}, \mathbf{x}_- \rangle + b = -1$$

In other words: we want our datapoints two lie at least a distance of 1 away from the decision hyperplane into both directions. To be more precise: for our positive examples (those with label $y_n = +1$) we want the following to hold: $\langle \mathbf{w}, \mathbf{x}_n \rangle + b \geq +1$.

For our negative examples (those with label $y_n = -1$) we want the opposite: $\langle \mathbf{w}, \mathbf{x}_n \rangle + b \leq -1$. This can be combined into a single equation: $y_n(\langle \mathbf{w}, \mathbf{x}_n \rangle + b) \geq 1$. This is our first goal: **We want a decision boundary that classifies our training examples correctly.**

2.2 Goal 2

Our second goal is to maximize the margin of this decision boundary. The margin is given by $\frac{1}{\|\mathbf{w}\|}$. If you would like to understand where this value is coming from take a look at the section “(Optional) Deriving the margin equation” below.

Our goal to maximize the margin can be expressed as follows:

$$\max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|}$$

Instead of maximizing $\frac{1}{\|\mathbf{w}\|}$ we can instead minimize $\frac{1}{2} \|\mathbf{w}\|^2$. This simplifies the computation of the gradient.

2.3 Combined goal

Combining goal one and goal two yields the following objective function:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

$$\text{subject to: } y_n(\langle \mathbf{w}, \mathbf{x}_n \rangle + b) \geq 1 \text{ for all } n = 1, \dots, N$$

In words: we want to find the values for $\mathbf{w}$ and b that maximize the margin while classifying all training examples correctly. This approach is called the *hard-margin support vector machine*. “Hard” because it does not allow for violations of the margin requirement (= no points are allowed to be within the margin).

2.4 (Optional) Deriving the margin equation

We can derive the width of the margin in several ways (see sections 12.2.1-12.2.2 of the [Mathematics for Machine Learning book](#)). Personally, I found the explanation of [this MIT lecture on SVMs](#) easiest to understand.

The derivation of the margin is based on the assumptions that we have already noted above:

$$\langle \mathbf{w}, \mathbf{x}_+ \rangle + b = +1$$

$$\langle \mathbf{w}, \mathbf{x}_- \rangle + b = -1$$

Including the label of each example we can rewrite this as

$$y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1 = 0$$

Let’s say we have a positive example $\mathbf{x}_+$ that lies on the right delimiting hyperplane and a negative example $\mathbf{x}_-$ that lies on the left delimiting hyperplane. The distance between these two vectors is given by $(\mathbf{x}_+ - \mathbf{x}_-)$. We want to compute the orthogonal projection of the vector onto the line that is perpendicular to the decision hyperplane. This would give us the width between the two delimiting hyperplanes. We can compute this by multiplying the vector $(\mathbf{x}_+ - \mathbf{x}_-)$ with a vector that is perpendicular to the hyperplane. We know that the vector $\mathbf{w}$ is perpendicular to the decision hyperplane. So we can compute the margin by multiplying $(\mathbf{x}_+ - \mathbf{x}_-)$ with the vector $\mathbf{w}$ where the latter is divided by the scale $\|\mathbf{w}\|$ to make it a unit vector.

x2↑

Search

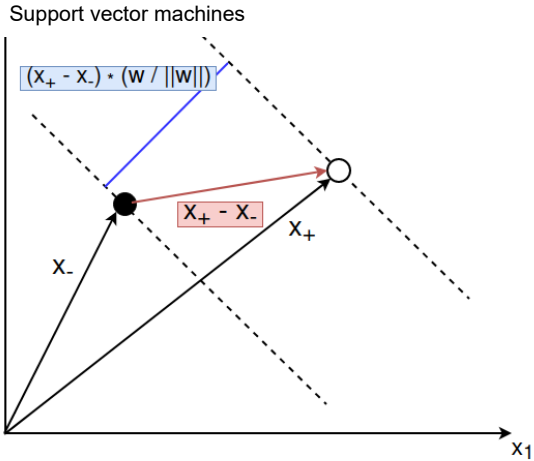
Posts

- Personal news
- My path to machine lec

Table of Contents

- 1. What are support vector machines?
- 2. Primal approach -

- + Books
- Machine Learning
 - Bayesian linear regression
 - Bayesian linear regression
 - KL Divergence
 - Principal component analysis
 - Support vector machines
 - Variational Inference
- + Python
- + Software Engineering



$$\begin{aligned} \text{width} &= (\mathbf{x}_+ - \mathbf{x}_-) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} \\ &= \frac{\mathbf{x}_+ \cdot \mathbf{w}}{\|\mathbf{w}\|} - \frac{\mathbf{x}_- \cdot \mathbf{w}}{\|\mathbf{w}\|} \end{aligned}$$

For the positive example $\mathbf{x}_+$ we have $y_+ = +1$ and therefore $(\langle \mathbf{w}, \mathbf{x}_+ \rangle = 1 - b$. For the negative example $\mathbf{x}_-$ we have $y_- = -1$ and therefore $-(\langle \mathbf{w}, \mathbf{x}_- \rangle) = 1 + b$:

$$\begin{aligned} \text{width} &= (\mathbf{x}_+ - \mathbf{x}_-) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} \\ &= \frac{\mathbf{x}_+ \cdot \mathbf{w}}{\|\mathbf{w}\|} - \frac{\mathbf{x}_- \cdot \mathbf{w}}{\|\mathbf{w}\|} \\ &= \frac{(1 - b) + (1 + b)}{\|\mathbf{w}\|} \\ &= \frac{2}{\|\mathbf{w}\|} \end{aligned}$$

We conclude that the width between the two delimiting hyperplanes equals $\frac{2}{\|\mathbf{w}\|}$. And therefore the distance from the decision hyperplane and each delimiting hyperplane is $\frac{1}{\|\mathbf{w}\|}$.

3. Primal approach - Soft-margin SVM

In most real-world situations the available data is not linearly separable. Even if it is, we might not want to separate the data well while ignoring some noisy examples and outliers. This motivated an SVM called *soft-margin SVM*.

A soft-margin SVM allows for violations of the margin requirement (= classification errors). In real-world examples need to be perfectly classified. They might fall within the margin or even lie on the wrong side of the decision hyperplane. However, such violations are not for free. We pay a cost for each violation, where the cost is proportional to how far the example is from meeting the margin requirement.

To implement this we introduce so called *slack variables* ξ_n . Each training example $(\mathbf{x}_n, y_n)$ is associated with a slack variable ξ_n . The slack variable allows this example to be within the margin or even on the wrong side of the decision hyperplane.

- If $\xi_n = 0$ the training example $(\mathbf{x}_n, y_n)$ lies exactly on the margin
- $0 < \xi_n < 1$ the training example lies within the margin but on the correct side of the decision hyperplane
- $\xi_n \geq 1$ the training example lies on the wrong side of the decision hyperplane

We extend our objective function to include the slack variables as follows:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n \\ \text{subject to:} \quad & y_n (\langle \mathbf{w}, \mathbf{x}_n \rangle + b) \geq 1 - \xi_n \\ & \xi_n \geq 0 \text{ for all } n = 1, \dots, N \end{aligned}$$

The parameter C is a regularization term that controls the trade-off between maximizing the margin and minimizing the training error (which in turn means classifying all training examples correctly). If the value of C is small, we care more about maximizing the margin than classifying all points correctly. If the value of C is large, we care more about classifying all points correctly than maximizing the margin.

4. Solving the primal optimization problem

Theoretically, the primal SVM can be solved in multiple ways. The most well known way is to use the [hinge loss function](#) together with [subgradient descent](#).

4.1 Hinge loss function

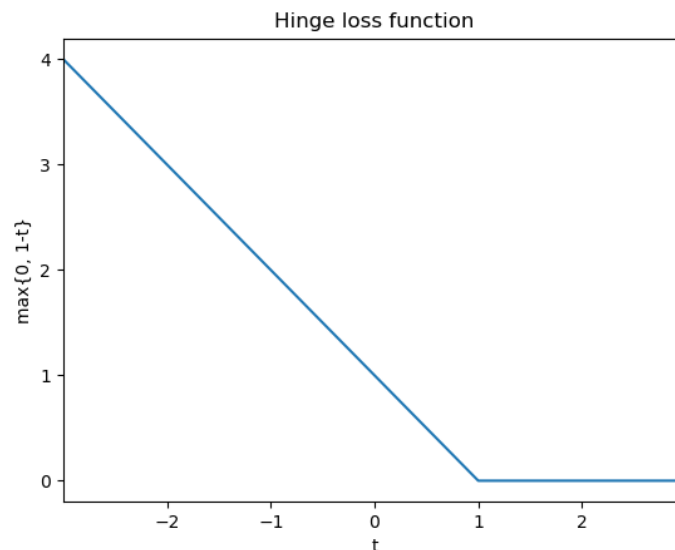
The hinge loss function given the true target $y \in \{-1, +1\}$ and the prediction $f(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b$ is computed as follows:

$$\ell(t) = \max(0, 1 - t) \quad \text{where} \quad t = y \cdot f(\mathbf{x}) = y \cdot (\langle \mathbf{w}, \mathbf{x} \rangle + b)$$

Let's understand the output of this loss function with a few examples:

- Hard-margin SVM
 - 2.1 Goal 1
 - 2.2 Goal 2
 - 2.3 Combined goal
 - 2.4 (Optional) Deriving the margin equation
- 3. Primal approach - Soft-margin SVM
- 4. Solving the primal optimization problem
 - 4.1 Hinge loss function
 - 4.2 Updated objective function
 - 4.3 [Optional] Three parts of the objective function
 - 4.4 Sub-gradient descent
 - 4.5 [Optional] Difference subgradient descent and gradient descent
 - 4.5.1 What is a subgradient?
 - 4.5.2 Subgradient method
- 5. Implementation primal approach
 - 5.1 Toy dataset
 - 5.2 SVM class definition
 - 5.3 Training and testing an SVM
 - 5.4 Visualizing the decision boundary
- 6. Dual approach
 - 6.1 Recap Lagrange multipliers
 - 6.2 Recap Lagrangian
 - 6.3 Dual optimization problem
- 7. Primal vs. dual approach
- 8. Kernels / non-linear SVM
 - 8.1 What is a kernel?
 - 8.2 What are kernels good for?
 - 8.3 Example
 - 8.4 Can we also use kernels in the primal SVM?
 - 8.5 How do I choose the right kernel for my problem?
- 9. Sources and further reading

- If a training example has label $y = -1$ and the prediction is on the correct side of the margin (that is, $f(\mathbf{x}) \leq -1$), the value of t is larger or equal to $+1$. Therefore, the hinge loss will be zero ($\ell(t) = 0$)
- The same holds if a training example has label $y = 1$ and the prediction is on the correct side of the margin (that is, $f(\mathbf{x}) \geq 1$)
- If a training example ($y = 1$) is on the correct side of the decision hyperplane but lies within the margin (that is, $0 < f(\mathbf{x}) < 1$) the hinge loss will output a positive value.
- If a training example ($y = 1$) is on the wrong side of the decision hyperplane (that is, $f(\mathbf{x}) < 0$), the hinge loss returns an even larger value. This value increases linearly with the distance from the decision hyperplane



4.2 Updated objective function

Using the hinge loss we can reformulate the optimization problem of the primal soft-margin SVM. Given a dataset $D = \{\mathbf{x}_n, y_n\}_{n=1}^N$ we would like to minimize the total loss which is now given by:

$$\min_{\mathbf{w}, b} \frac{1}{2} |\mathbf{w}|^2 + C \sum_{n=1}^N \max \{0, 1 - y_n (\langle \mathbf{w}, \mathbf{x}_n \rangle + b)\}$$

If you would like to understand why this is equivalent to our previous formulation of the soft-margin SVM please take a look at chapter 12.2.5 of the [Mathematics for Machine Learning book](#).

4.3 [Optional] Three parts of the objective function

Our objective function can be divided into three distinct parts:

Part 1: $\frac{1}{2} |\mathbf{w}|^2$

This part is also called the *regularization term*. It expresses a preference for solutions that separate the datapoints well, thereby maximizing the margin. In theory, we could replace this term by a different regularization term that expresses a different preference.

Part 2: $\sum_{n=1}^N \max \{0, 1 - y_n (\langle \mathbf{w}, \mathbf{x}_n \rangle + b)\}$

This part is also called the *empirical loss*. In our case it's the hinge loss which penalizes solutions that make mistakes when classifying the training examples. In theory, this term could be replaced with another loss function that expresses a different preference.

Part 3: The hyperparameter C that controls the tradeoff between a large margin and a small hinge loss.

4.4 Sub-gradient descent

The hinge loss function is not differentiable (namely at the point $t = 1$). Therefore, we cannot compute the gradient right away. However, we can use a method called [subgradient descent](#) to solve our optimization problem. To simplify the derivation we will adapt two things:

1. We assume that the bias b is contained in our weight vector as the first entry w_0 , that is $\mathbf{w} = [b, w_1, \dots, w_D]$
2. We divide the hinge loss by the number of samples

Our cost function is then given by

$$J(\mathbf{w}) = \frac{1}{2} |\mathbf{w}|^2 + C \frac{1}{N} \sum_{n=1}^N \max \{0, 1 - y_n (\langle \mathbf{w}, \mathbf{x}_n \rangle)\}$$

We will reformulate this to simplify computing the gradient:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \left\{ \frac{1}{2} |\mathbf{w}|^2 + C \max \{0, 1 - y_n (\langle \mathbf{w}, \mathbf{x}_n \rangle)\} \right\}$$

The gradient is given by:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \begin{cases} \mathbf{w} & \text{if } \max(0, 1 - y_n (\langle \mathbf{w}, \mathbf{x}_n \rangle)) = 0 \\ \mathbf{w} - C y_n \mathbf{x}_n & \text{otherwise} \end{cases}$$

With this formula we can apply stochastic gradient descent to solve the optimization problem.

4.5 [Optional] Difference subgradient descent and gradient descent

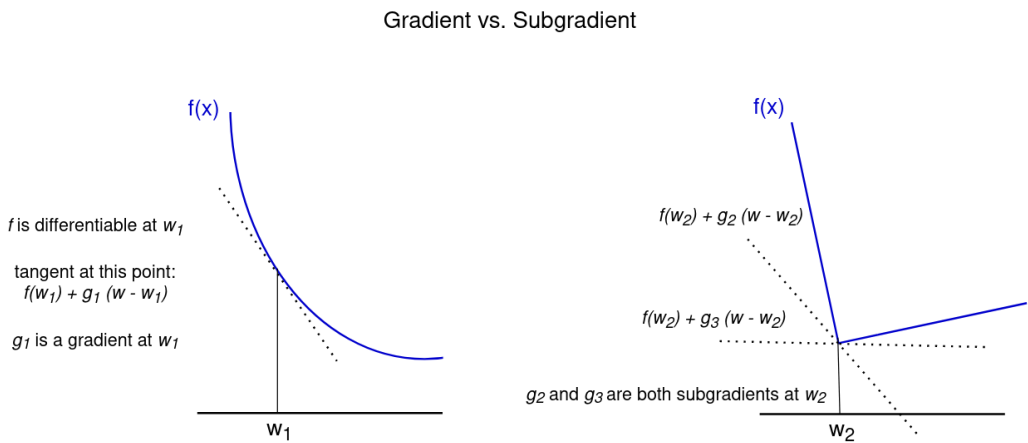
The subgradient method allows us to minimize a non-differentiable convex function. Although looking similar to gradient descent the method has several important differences.

4.5.1 What is a subgradient?

A subgradient can be described as a generalization of gradients to non-differentiable functions. Informally, a sub-tangent at a point is any line that lies below the function at the point. The subgradient is the slope of this line. Formally, the subgradient a convex function f at w_0 is defined as all vectors g such that for any other point w

$$f(w) - f(w_0) \geq g \cdot (w - w_0)$$

If f is differentiable at w_0 , the subgradient contains only one vector which equals the gradient $\nabla f(w_0)$. If, however, f is not differentiable, there may be several values for g that satisfy this inequality. This is illustrated in the figure below.



4.5.2 Subgradient method

To minimize the objective function f the subgradient method uses the following update formula for iteration $k + 1$:

$$w^{(k+1)} = w^{(k)} - \alpha_k g^{(k)}$$

Where $g^{(k)}$ is *any* subgradient of f at $w^{(k)}$ and α_k is the k -th step size. Thus, at each iteration, we make a step into the direction of the negative subgradient. When f is differentiable, $g^{(k)}$ equals the gradient $\nabla f(x^{(k)})$ and the method reduces to the standard gradient descent method.

More details on the subgradient method can be found [here](#).

5. Implementation primal approach

5.1 Toy dataset

To implement what we have learned about primal SVMs we first have to generate a dataset. In the cell below we create a simple dataset with two features and labels +1 and -1. We further split the dataset into a test and train set.

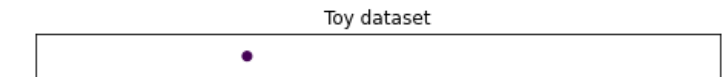
```
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

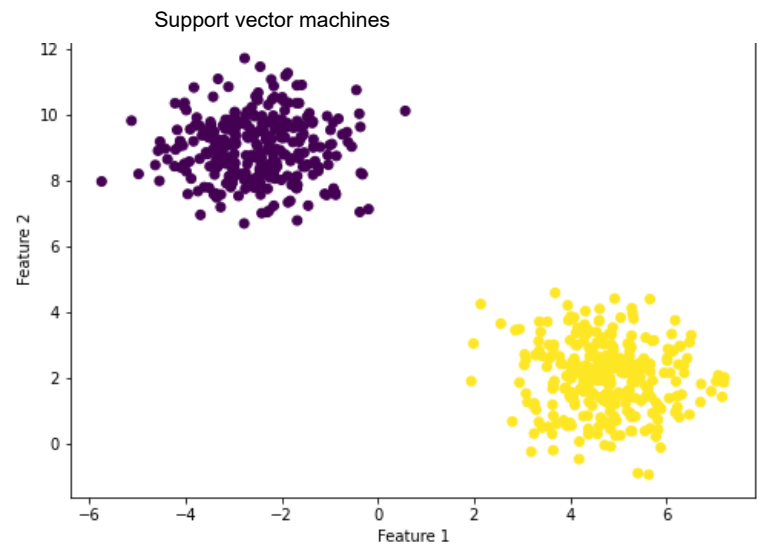
features, targets = make_blobs(n_samples=600, centers=2, n_features=2, random_state=42)

# The function outputs targets 0 and 1 so we need to convert targets 0 to -1
transformed_targets = [-1 if t == 0 else +1 for t in targets]

# Plot the dataset
plt.figure(figsize=(8, 6))
plt.scatter(features[:, 0], features[:, 1], c = transformed_targets)
plt.title("Toy dataset")
plt.ylabel("Feature 2")
plt.xlabel("Feature 1")
plt.show()

# Split data into training and test set
features_train, features_test, labels_train, labels_test = train_test_split(features,
                                                                              transformed_targets,
                                                                              test_size = 0.3)
```





5.2 SVM class definition

Next, we would like to implement an SVM class. We will use the knowledge we already acquired:

- 1. Our objective function using the hinge loss function is given by:

$$J(\mathbf{w}) = \frac{1}{2}|\mathbf{w}|^2 + C\frac{1}{N}\sum_{n=1}^N\max\{0, 1 - y_n(\langle\mathbf{w}, \mathbf{x}_n\rangle)\}$$

- 2. We can minimize this function by computing the gradient:

$$\nabla_w J(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^N\begin{cases} \mathbf{w} & \text{if } \max(0, 1 - y_n(\langle\mathbf{w}, \mathbf{x}_n\rangle)) = 0 \\ \mathbf{w} - Cy_n\mathbf{x}_n & \text{otherwise} \end{cases}.$$

- 3. Given the gradient we use stochastic gradient descent to train our model
- 4. After training our model we can make predictions using the [sign function](#)

As mentioned previously, we will assume that the bias b is contained in our weight vector as the first entry w_0 , that is $\mathbf{w} = [b, w_1, \dots, w_D] = \mathbf{w} = [w_0, w_1, \dots, w_D]$.

```
import numpy as np
from sklearn.utils import shuffle
```

```

class LinearSVM:

    def __init__(self, regularization_param):
        """
        Initialize the model by setting the regularization parameter
        and a boolean variable for our trained weights.
        """
        self.regularization_param = regularization_param
        self.trained_weights = None

    def add_bias_term(self, features):
        """
        Add intercept 1 to each training example for bias b
        """
        n_samples = features.shape[0]
        ones = np.ones((n_samples, 1))
        return np.concatenate((ones, features), axis=1)

    def compute_cost(self, weights, features, labels) -> float:
        """
        Compute the value of the cost function
        """
        n_samples = features.shape[0]

        # Compute hinge loss
        predictions = np.dot(features, weights).flatten()
        distances = 1 - labels * predictions
        hinge_losses = np.maximum(0, distances)

        # Compute sum of the individual hinge losses
        sum_hinge_loss = np.sum(hinge_losses) / n_samples

        # Compute entire cost
        cost = (1 / 2) * np.dot(weights.T, weights) + self.regularization_param * sum_hinge_loss

        return float(cost)

    def compute_gradient(self, weights, features, labels) -> np.ndarray:
        """
        Compute the gradient, needed for training
        """
        predictions = np.dot(features, weights)
        distances = 1 - labels * predictions
        n_samples, n_feat = features.shape
        sub_gradients = np.zeros((1, n_feat))

        for idx, dist in enumerate(distances):
            if max(0, dist) == 0:
                sub_gradients += weights.T
            else:
                sub_grad = weights.T - (self.regularization_param * features[idx] * labels[idx])
                sub_gradients += sub_grad

        # Sum up and divide by the number of samples
        avg_gradient = sum(sub_gradients) / len(labels)

        return avg_gradient

    def train(self, train_features, train_labels, n_epochs, learning_rate=0.01, batch_size=1):
        """
        Train the model with stochastic gradient descent using the
        specified number of epochs, learning rate and batch size.
        """
        # Add bias term to features
        train_features = self.add_bias_term(train_features)

        # Initialize weight vector
        n_samples, n_feat = train_features.shape
        weights = np.zeros(n_feat)[: , np.newaxis]

        # Train the model for a certain number of epochs
        for epoch in range(n_epochs):
            features, labels = shuffle(train_features, train_labels)
            features, labels = train_features, train_labels
            start, end = 0, batch_size
            while end <= len(labels): # Training loop over the dataset
                batch = features[start:end]
                batch_labels = labels[start:end]

                grad = self.compute_gradient(weights, batch, batch_labels)
                update = (learning_rate * grad)[: , np.newaxis]
                weights = weights - update
                start, end = end, end + batch_size

```

```
        current_cost = self.compute_cost(weights, features, labels)
        print(f"Epoch {epoch + 1}, cost: {current_cost}")

    # Set the trained weights to allow making predictions
    self.trained_weights = weights

def predict(self, test_features) -> np.ndarray:
    """
    Predict labels for new test features.
    Raises ValueError if model has not been trained yet.
    """
    test_features = self.add_bias_term(test_features)
    if self.trained_weights is None:
        raise ValueError("You haven't trained the SVM yet!")

    predicted_labels = []
    n_samples = test_features.shape[0]
    for idx in range(n_samples):
        prediction = np.sign(np.dot(self.trained_weights.T, test_features[idx]))
        predicted_labels.append(prediction)

    return np.array(predicted_labels)

# Compute some values to make sure the cost is computed correctly
# I calculated the values for this example by hand first
svm = LinearSVM(regularization_param=1)
weights = np.array([1, 2])[ :, np.newaxis]
features = np.array([[0.5], [2.5]])
new_features = svm.add_bias_term(features)

labels = np.array([-1, +1])
assert svm.compute_cost(weights, new_features, labels) == 4.
gradient = svm.compute_gradient(weights, new_features, labels[:, np.newaxis])
```

5.3 Training and testing an SVM

After defining our SVM class we can train a model and test it on unseen examples.

```
# Initialize a new SVM and train it on the given toy dataset
regularization_param = 100
lr = 0.000001
svm = LinearSVM(regularization_param)
trained_weights = svm.train(features_train, labels_train, n_epochs=10, learning_rate=lr)

Epoch 1, cost: 28.999523042690537
Epoch 2, cost: 8.626987624824444
Epoch 3, cost: 3.8253539901075273
Epoch 4, cost: 2.4947875220684446
Epoch 5, cost: 1.9543704229309797
Epoch 6, cost: 1.5987135095492675
Epoch 7, cost: 1.3611077210579672
Epoch 8, cost: 1.1702119735961691
Epoch 9, cost: 1.0372585586110663
Epoch 10, cost: 0.9557095984777207

# Predict Lables for unknown test samples
from sklearn.metrics import accuracy_score, recall_score, precision_score

predicted_labels = svm.predict(features_test)
predicted_labels = predicted_labels.flatten()

print("Accuracy on test dataset: {}".format(accuracy_score(labels_test, predicted_labels)))
print("Recall on test dataset: {}".format(recall_score(labels_test, predicted_labels)))
print("Precision on test dataset: {}".format(recall_score(labels_test, predicted_labels)))

Accuracy on test dataset: 1.0
Recall on test dataset: 1.0
Precision on test dataset: 1.0
```

5.4 Visualizing the decision boundary

Given our trained model we can visualize the decision boundary, as done below.

```
import numpy as np

# Create dataset for visualization
```



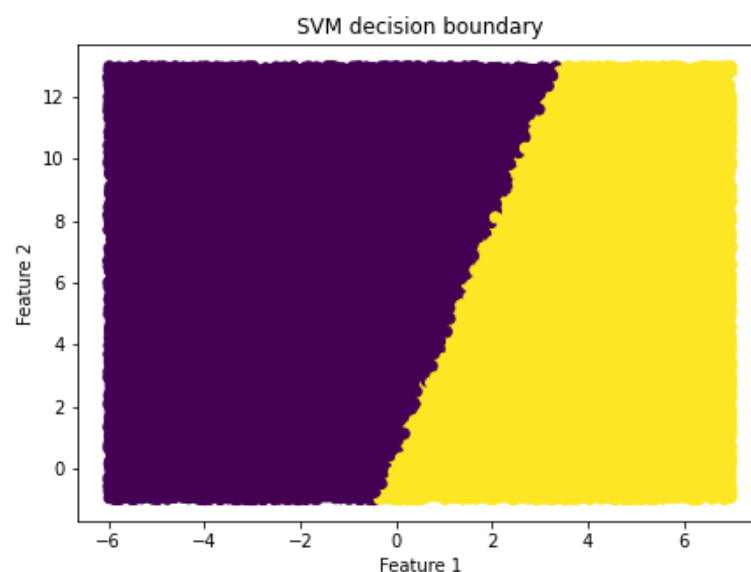
```

size=40000
feat_1 = np.random.uniform(low=-6, high=7, size=size)
feat_2 = np.random.uniform(low=-1, high=13, size=size)
features_vis = np.column_stack((feat_1, feat_2))

labels_vis = svm.predict(features_vis)

# Plot the decision boundary
plt.figure(figsize=(7, 5))
plt.scatter(features_vis[:, 0], features_vis[:, 1], c = labels_vis)
plt.title("SVM decision boundary")
plt.ylabel("Feature 2")
plt.xlabel("Feature 1")
plt.show()

```



6. Dual approach

In the previous sections we took a detailed look at the primal SVM. To solve a primal SVM, we need to find the best values for the weights and bias. Recall that our input examples $\mathbf{x} \in \mathbb{R}^D$ have D features. Consequently, our weights $\mathbf{w}$ have D features, too. This can become problematic if the number of features D is large.

That's where the second way of formalizing SVMs (called *dual approach*) comes in handy. The optimization problem of the dual approach is independent of the number of features. Instead, the number of parameters increases with the number of examples in the training set.

The dual approach uses the method of Lagrange multipliers. Lagrange multipliers allow us to find the minimum or maximum of a function if there are one or more constraints on the input values we are allowed to use.

If you never heard of Lagrange multipliers I can recommend the [blog posts](#) and [video tutorials](#) on the topic from Khan Academy.

6.1 Recap Lagrange multipliers

Lagrange multipliers allow us to solve constrained optimization problems. Let's say we want to maximize the function $f(x, y) = 2x + y$ under the constraint that our values of x and y satisfy the following equation: $g(x, y) = x^2 + y^2 = 1$. This constraint equation describes a circle of radius 1.

The key insight behind the solution to this problem is that we need to find those values for x and y where the gradients of f and g are aligned. This can be expressed using a Lagrange multiplier (typically λ):

We want to find those values x_m, y_m where $\nabla f(x_m, y_m) = \lambda \nabla g(x_m, y_m)$.

In our example the gradient vectors look as follows:

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial}{\partial x}(2x + y) \\ \frac{\partial}{\partial y}(2x + y) \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\nabla g(x, y) = \begin{bmatrix} \frac{\partial}{\partial x}(x^2 + y^2 - 1) \\ \frac{\partial}{\partial y}(x^2 + y^2 - 1) \end{bmatrix} = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$$

Therefore, the tangency condition results in

$$\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \lambda \begin{bmatrix} 2x_m \\ 2y_m \end{bmatrix}$$

We can rewrite the vector form into individual equations that can be solved by hand:

- $2 = \lambda 2x_m$
- $1 = \lambda 2y_m$
- $x_m^2 + y_m^2 = 1$

Solving the equations yields

$$\begin{aligned}(x_0, y_0) &= \left(\frac{1}{\lambda_0}, \frac{1}{2\lambda_0} \right) \\ &= \left(\frac{2}{\sqrt{5}}, \frac{1}{\sqrt{5}} \right) \quad \text{or} \quad \left(\frac{-2}{\sqrt{5}}, \frac{-1}{\sqrt{5}} \right)\end{aligned}$$

where the first point denotes a maximum (what we wanted to find) and the second a minimum. This solves our constrained optimization problem. For more details and a full solution look at [this Khan academy post](#).

6.2 Recap Lagrangian

The Lagrangian is a way to repackage the individual conditions of our constrained optimization problem into a single equation. In the example above we wanted to optimize some function $f(x, y)$ under the constraint that the inputs x and y satisfy the equation $g(x, y) = x^2 + y^2 = c$. In our case the constant c was given by 1. We know that the solution is given by those points where the gradients of f and g align. The Lagrangian function puts all of this into a single equation:

$$\mathcal{L}(x, y, \lambda) = f(x, y) - \lambda(g(x, y) - c)$$

When computing the partial derivatives of $\mathcal{L}$ with respect to x, y and λ and setting them to zero, we will find that they correspond exactly to the three constraints we looked at earlier. This can be summarized by simply setting the gradient of $\mathcal{L}$ equal to the zero vector: $\nabla \mathcal{L} = \mathbf{0}$. The compact Lagrangian form is often used when solving constrained optimization problem with computers because it summarizes the elaborate problem with multiple constraints into a single equation. For more details and a full solution look at [this Khan academy post](#).

6.3 Dual optimization problem

For the primal soft-margin SVM we considered the following optimization problem:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n$$

subject to:

$$y_n(\langle \mathbf{w}, \mathbf{x}_n \rangle + b) \geq 1 - \xi_n$$

$$\xi_i > 0 \text{ for all } i = 1, \dots, N$$

To derive the corresponding Lagrangian we will introduce two Lagrange multipliers: α_n for the first constraint (that all examples are classified correctly) and λ_n for the second constraint (non-negativity of the slack variables). The Lagrangian is then given by:

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b, \xi, \alpha, \gamma) &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n \\ &\quad - \underbrace{\sum_{n=1}^N \alpha_n (y_n (\langle \mathbf{w}, \mathbf{x}_n \rangle + b) - 1 + \xi_n)}_{\text{first constraint}} - \underbrace{\sum_{n=1}^N \gamma_n \xi_n}_{\text{second constraint}}\end{aligned}$$

Next, we have to compute the partial derivatives of the Lagrangian with respect to the variables $\mathbf{w}, b$ and ξ : $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \frac{\partial \mathcal{L}}{\partial b}, \frac{\partial \mathcal{L}}{\partial \xi}$.

When setting the first partial derivative to zero we obtain an important interim result:

$$\mathbf{w} = \sum_{n=1}^N \alpha_n y_n \mathbf{x}_n$$

This equation states the the optimal solution for the weight vector is given by a linear combination of our training examples. After setting the other partial derivatives to zero, using the result and simplifying the equations we end up with the following optimization problem (for details see section 12.3.1 of the [Mathematics for Machine Learning book](#)):

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle - \sum_{i=1}^N \alpha_i$$

subject to:

$$\sum_{i=1}^N y_i \alpha_i = 0$$

$$0 \leq \alpha_i \leq C \text{ for all } i = 1, \dots, N$$

This constrained quadratic optimization problem can be solved very efficiently, for example with quadratic programming techniques. One popular library for solving dual SVMs is [libsvm](#) which makes use of a decomposition method to solve the problem (see [this paper](#) for more details). However, several other approaches exist.

7. Primal vs. dual approach

Most SVM research in the last decade has been about the dual formulation. Why this is the case is not clear. Both approaches

have advantages and disadvantages. In the paper [“Training a Support Vector Machine in the Primal”](#) Chapelle et al. mention the following hypothesis:

We believe that it is because SVMs were first introduced in their hard margin formulation (Boser et al., 1992), for which a dual optimization (because of the constraints) seems more natural. In general, however, soft margin SVMs should be preferred, even if the training data are separable: the decision boundary is more robust because more training points are taken into account (Chapelle et al., 2000). We do not pretend that primal optimization is better in general; our main motivation was to point out that primal and dual are two sides of the same coin and that there is no reason to look always at the same side.

8. Kernels / non-linear SVM

8.1 What is a kernel?

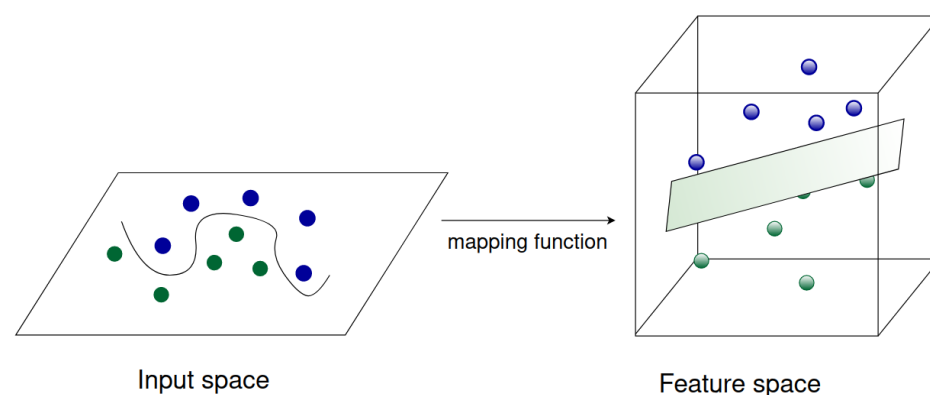
If you take another look at the optimization equation of dual SVMs you will notice that it computes the inner product $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ between all datapoints $\mathbf{x}_i, \mathbf{x}_j$. A kernel is a way to compute this inner product implicitly in some (potentially very high dimensional) feature space. To be more precise: assume we have some mapping function φ which maps an n dimensional input vector to an m dimensional output vector: $\varphi: \mathbb{R}^n \rightarrow \mathbb{R}^m$. Given this mapping function we can compute the dot product of two vectors $\mathbf{x}$ and $\mathbf{y}$ in this space as follows: $\varphi(\mathbf{x})^T \varphi(\mathbf{y})$.

A kernel is a function k that gives the same result as this dot product: $k(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$. In other words: the kernel function is equivalent to the dot product of the mapping function.

8.2 What are kernels good for?

Until now (apart from the soft-margin SVM) our SVMs, both primal and dual, are only able to classify data that is [linearly separable](#). However, most datasets in practice won't be of this form. We need a way to classify data that is **not** linearly separable. This is where the so called **kernel trick** comes into play.

Because the objective function of the dual SVM contains inner products only between datapoints $\mathbf{x}_i, \mathbf{x}_j$, we can easily replace this inner product (that is, $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$) with some mapping function $\varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$. This mapping function can be non-linear, allowing us to compute an SVM that is non-linear with respect to the input examples. The mapping function takes our input data (which is not linearly separable) and transforms it into some higher-dimensional space where it becomes linearly separable. This is illustrated in the figure below.



In theory, we could use any mapping function we like. In practice, however, computing inner products is expensive. Therefore, we use mapping functions that have a corresponding kernel function. This will allow us to map the datapoints into a higher dimensional space without ever explicitly computing the (expensive) inner products.

Let's take a look at an example.

8.3 Example

Note: this example was taken from [this StackExchange post](#).

We can create a simple polynomial kernel as follows: $k(\mathbf{x}, \mathbf{y}) = (1 + \mathbf{x}^T \mathbf{y})^2$ with $\mathbf{x}, \mathbf{y} \in \mathbb{R}^2$. The kernel does not seem to correspond to any mapping function φ , it's just a function that returns a real number. Our input vectors $\mathbf{x}, \mathbf{y}$ are 2-dimensional: $\mathbf{x} = (x_1, x_2)$ and $\mathbf{y} = (y_1, y_2)$. With this knowledge we can expand the kernel computation:

$$\begin{aligned} k(\mathbf{x}, \mathbf{y}) &= (1 + \mathbf{x}^T \mathbf{y})^2 \\ &= (1 + x_1 y_1 + x_2 y_2)^2 \\ &= 1 + x_1^2 y_1^2 + x_2^2 y_2^2 + 2x_1 y_1 + 2x_2 y_2 + 2x_1 x_2 y_1 y_2 \end{aligned}$$

Note that this is nothing else but a dot product between two vectors $(1, x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1 x_2)$ and $(1, y_1^2, y_2^2, \sqrt{2}y_1, \sqrt{2}y_2, \sqrt{2}y_1 y_2)$. This can be expressed with the following mapping function:

$$\varphi(\mathbf{x}) = \varphi(x_1, x_2) = (1, x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1 x_2)$$

So the kernel $k(\mathbf{x}, \mathbf{y}) = (1 + \mathbf{x}^T \mathbf{y})^2 = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$ computes a dot product in 6-dimensional space without explicitly visiting this space. The generalization from an inner product to a kernel function is known as the **kernel trick**.

Several popular kernel functions exist. Popular ones are, for example, the [polynomial kernel](#) or [RBF kernel](#).

8.4 Can we also use kernels in the primal SVM?

Yes, the kernel trick can be applied to primal SVM's, too. It's not as straightforward as with dual SVMs but still possible.

Consider [this paper by Chapelle et al.](#) as an example.

8.5 How do I choose the right kernel for my problem?

The problem of choosing the right kernel has been answered in [this StackExchange post](#).

Summary:

- Without expert knowledge, the Radial Basis Function kernel makes a good default kernel (in case you need a non-linear model)
- The choice of the kernel and parameters can be automated by optimising a cross-validation based model selection
- Choosing the kernel and parameters automatically is tricky, as it is very easy to overfit the model selection criterion

9. Sources and further reading

The basis for this notebook is chapter 12 of the book [Mathematics for Machine Learning](#). I can highly recommend to read through the entire chapter to get a deeper understanding of support vector machines.

Another source I liked very much is [this MIT lecture on SVMs](#).

[🔗 Improve this page](#)

◀ Prev

Principal component analysis (PCA)

Next ▶

Personal reading list of non-fiction books

Navigation

- [About](#)
- [Skills](#)
- [Experiences](#)
- [Education](#)
- [Projects](#)
- [Talks & Podcasts](#)
- [Recent Posts](#)

Contact me:

Email: popkes@gmx.net



© 2020-2021 Copyright.

Powered by 